

## **2.4. Transformaciones M2M**

Las transformaciones modelo-a-modelo (M2M) toman un modelo de origen y producen un modelo destino, ambos conformes a sus respectivos metamodelos; dentro de MDA, el caso típico es el paso de un PIM a un PSM. El estándar del OMG que rige este tipo de transformación es QVT (Query/View/Transformation) v1.3 [1]. No obstante, la evidencia empírica reciente indica que la adopción industrial de lenguajes de transformación dedicados (MTL) como QVT o ATL sigue siendo limitada frente a lo que cabría esperar de un estándar consolidado: el estudio de Höppner et al. [2], basado en 56 entrevistas a investigadores y profesionales, documenta que la falta de tooling maduro y de evidencia sólida sobre sus ventajas frena su uso frente a lenguajes de propósito general.

Esta brecha entre el estándar y la práctica industrial explica un patrón frecuente en las herramientas MDD: muchas no exponen una etapa M2M diferenciada ni un lenguaje de transformación independiente conforme a QVT, sino que absorben esa lógica dentro del propio generador de código, traduciendo el modelo de origen directamente a texto sin materializar un PSM intermedio como artefacto propio. Esto no invalida la distinción conceptual entre M2M y M2T: evidencia, más bien, que en la práctica ambas etapas suelen fusionarse cuando el objetivo final es la generación de código y no la interoperabilidad entre modelos.

## **2.5. Transformaciones M2T**

Las transformaciones modelo-a-texto (M2T) generan artefactos textuales código fuente, configuración o documentación a partir de un modelo conforme a un metamodelo; el estándar OMG de referencia es MOFM2T v1.0 [3]. Un ejemplo reciente de esta lógica se encuentra en el trabajo de Jiménez-Navajas et al. [4], quienes implementan transformaciones M2T con el lenguaje EGL (Epsilon Generation Language) para producir código Python y Qiskit a partir de modelos UML extendidos, lo que evidencia cómo un motor M2T recorre los elementos del modelo y los inyecta en plantillas parametrizadas para producir texto ejecutable.

Las herramientas MDD implementan esta lógica de dos formas distintas. Por un lado, motores declarativos como Acceleo o EGL exponen la plantilla como un archivo de texto editable que sigue, con distinto grado de fidelidad, la gramática propuesta por MOFM2T. Por otro lado, un grupo amplio de herramientas comerciales integra la generación como un módulo o plug-in compilado que lee las clases, atributos, relaciones y operaciones del modelo y las vuelca en código siguiendo plantillas fijas embebidas en el propio generador, sin exponer una gramática pública ni permitir su edición directa; esta segunda variante limita la personalización a los parámetros que la propia herramienta expone.

## 2.8. Plantilla / template

Una plantilla o template es el fragmento parametrizado que la herramienta MDD instancia por cada elemento del modelo para producir el código correspondiente. Este mecanismo puede implementarse como archivo de texto editable, ligado a una gramática de tipo MOFM2T, o puede empaquetarse como un componente compilado que aplica la definición del modelo sin exponerla como texto. Paolone et al. [5] documentan este segundo caso con xGenerator, una plataforma MDA propietaria que genera aplicaciones Java MVC completas a partir de un modelo UML sin exponer la plantilla al desarrollador, tratándola como una caja negra cuya salida solo puede evaluarse por la calidad del código resultante y no por inspección directa de la plantilla.

Un mecanismo de parametrización habitual en este tipo de generadores es el bloque protegido: toda operación cuyo cuerpo se delimita mediante marcadores reservados conserva, en regeneraciones sucesivas, el contenido que el desarrollador haya escrito manualmente, evitando que una nueva ejecución del generador sobrescriba la lógica de negocio ya implementada. Este mecanismo es coherente con la lógica M2T y con el estándar MOFM2T [3], y constituye la base técnica que permite un ciclo de roundtrip controlado: el modelo puede regenerarse repetidamente sin perder el trabajo manual, siempre que este se mantenga dentro de los delimitadores correspondientes.

## Referencias

- [1] Object Management Group, “Meta Object Facility (MOF) 2.0 Query/View/Transformation Specification,” Needham, MA, Jun. 2016. Accessed: Jul. 18, 2026. [Online]. Available: <https://www.omg.org/spec/QVT/1.3/PDF>
- [2] S. Höppner, Y. Haas, M. Tichy, and K. Juhnke, “Advantages and disadvantages of (dedicated) model transformation languages: A qualitative interview study,” *Empir. Softw. Eng.*, vol. 27, no. 6, Nov. 2022, doi: 10.1007/s10664-022-10194-7.
- [3] Object Management Group, “MOF Model to Text Transformation Language,” Needham, MA, Jan. 2008. Accessed: Jul. 18, 2026. [Online]. Available: <https://www.omg.org/spec/MOFM2T/1.0/PDF>
- [4] L. Jiménez-Navajas, R. Pérez-Castillo, and M. Piattini, “Code generation for classical-quantum software systems modeled in UML,” *Softw. Syst. Model.*, vol. 24, no. 3, pp. 795–821, Jun. 2025, doi: 10.1007/s10270-024-01259-w.
- [5] G. Paolone, R. Paesani, M. Marinelli, and P. Di Felice, “Empirical Assessment of the Quality of MVC Web Applications Returned by xGenerator,” 2021, doi: 10.3390/computers.

